

What is Git?

- Git is the version control software.

What is GitHub?

- GitHub is where you host your projects online.

How to create git account in windows?

- The "**Git** account" is actually a **GitHub** account, which is a popular web-based hosting service for Git repositories.
- To start, you will create a free account on the GitHub website.

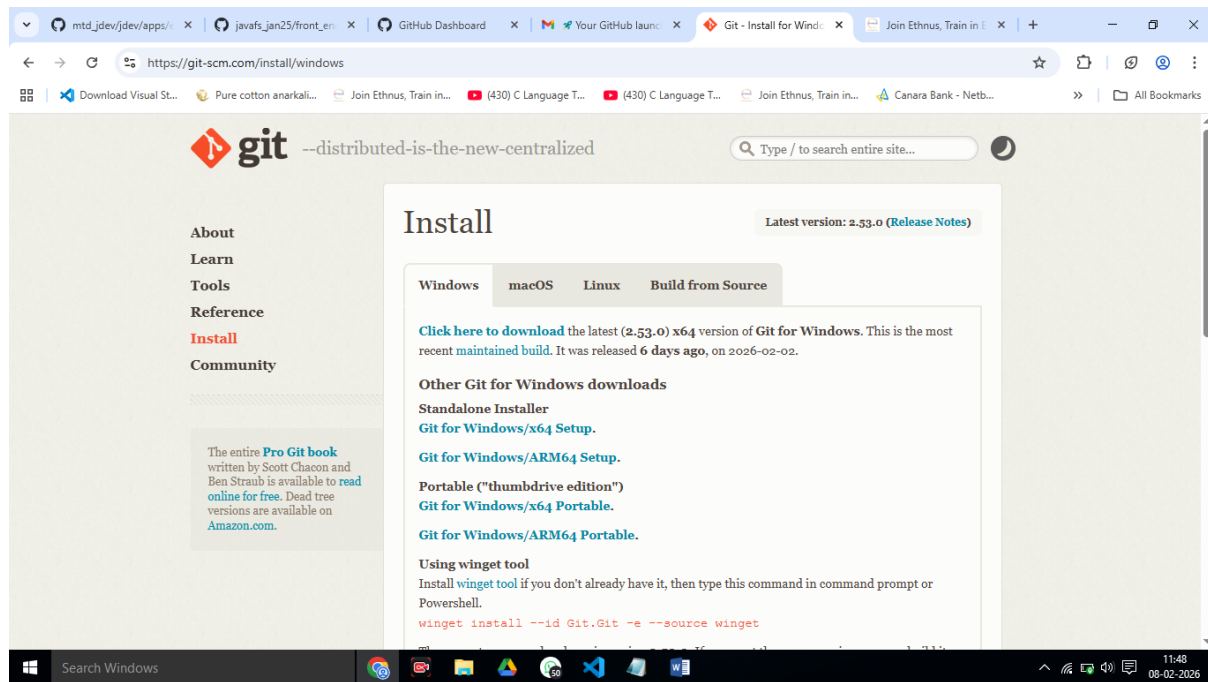
Step 1: Create a GitHub account

1. **Navigate to the GitHub** website in your browser by going to **<https://github.com>**.
2. Click the **Sign up** button in the top-right corner of the page.
3. Enter your **email** address and click **Continue**.
4. Create a strong password for your account and a unique username. Your username will be publicly visible and appear on your projects.
5. Answer the on-screen prompts and solve the **puzzle** to verify that you are not a robot.
6. A **launch code** will be sent to the email address you provided. Enter this code to verify your account.
7. After confirming, you will be asked a few questions to tailor your experience. You can choose a **free plan** and skip any personalization questions.

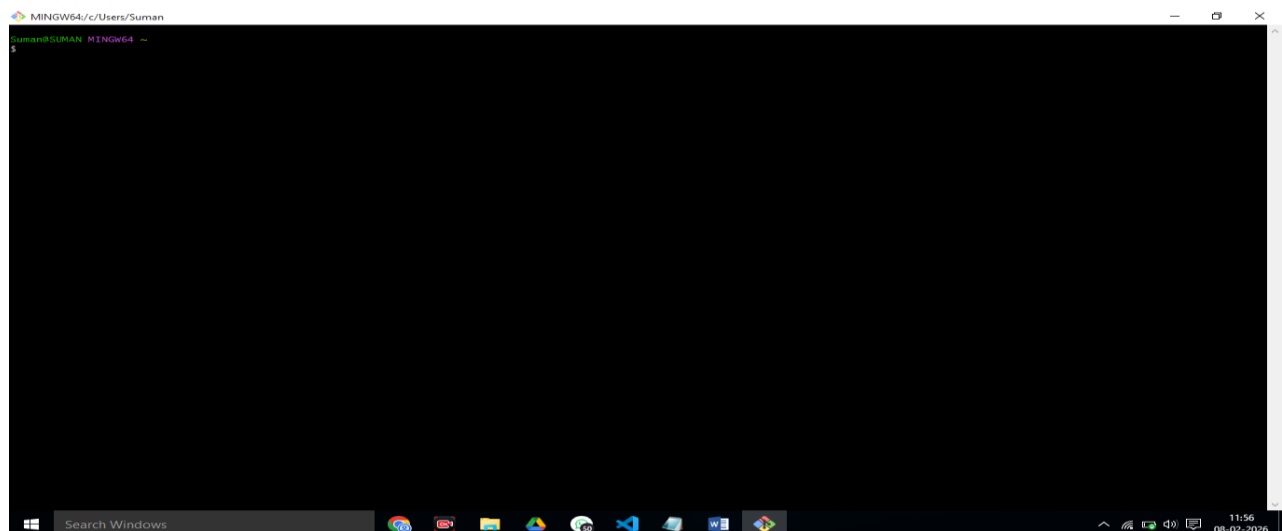
Step 2: Install Git

To use your GitHub account with your local Windows PC, you need to install the Git command-line tools.

1. Go to the official Git website at **<https://git-scm.com/download/win>**.
2. Click the link to download the latest version for **Windows**.



- 3.
4. Run the downloaded installer and follow the prompts. The default options are generally recommended for most users.
5. Before clicking the Finish button -> select Launch Git Bash and click finish button.
6. The Git installer includes a tool called **Git Bash**, which provides a Unix-like command-line interface for running Git commands on Windows.
7. GIT Bash .



- 8.

Step 3: Configure Git with your account

1. After installation, open **Git Bash** from your Start Menu.
 2. Set your **username** and **email** address. This information will be attached to your commits so they are associated with your GitHub account.
- Type the following command, replacing the name with your own:

```
$ git config --global user.name "Your Name"
```

- Type the next command, replacing the email with your own:

```
$ git config --global user.email "your_email@example.com"
```

=====

Step 4: Use your GitHub account

You can now start using Git on your Windows 10 PC and connect it to your new GitHub account.

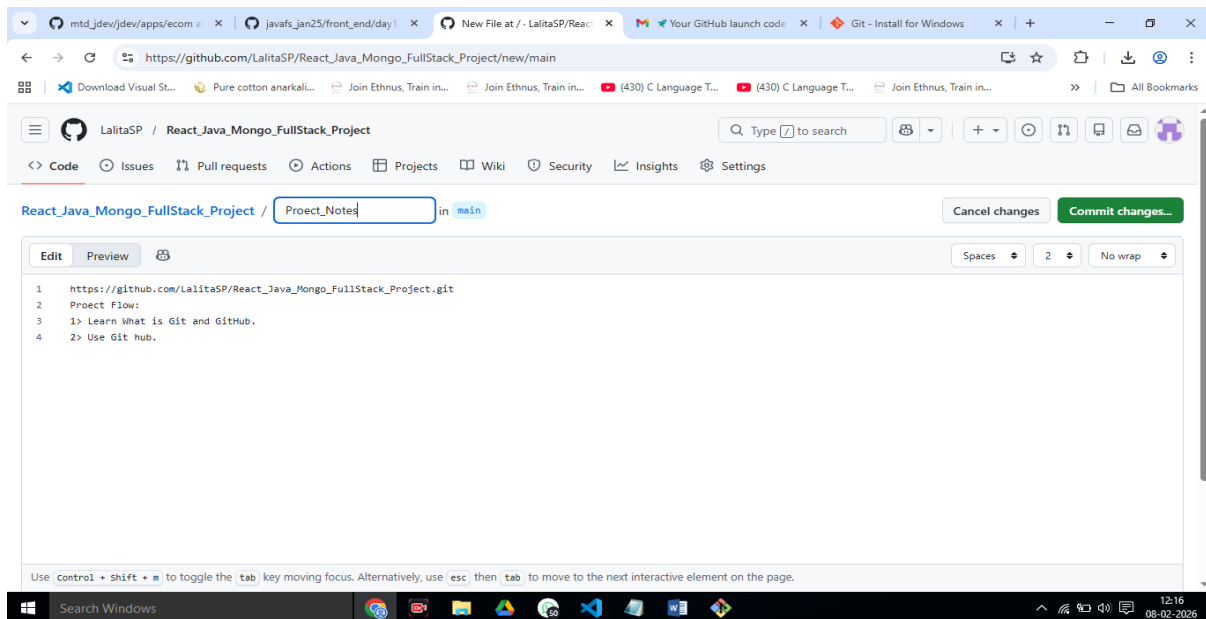
- **Create a new project:**

1> **Create a new repository on GitHub.com and then clone it to your computer.** or

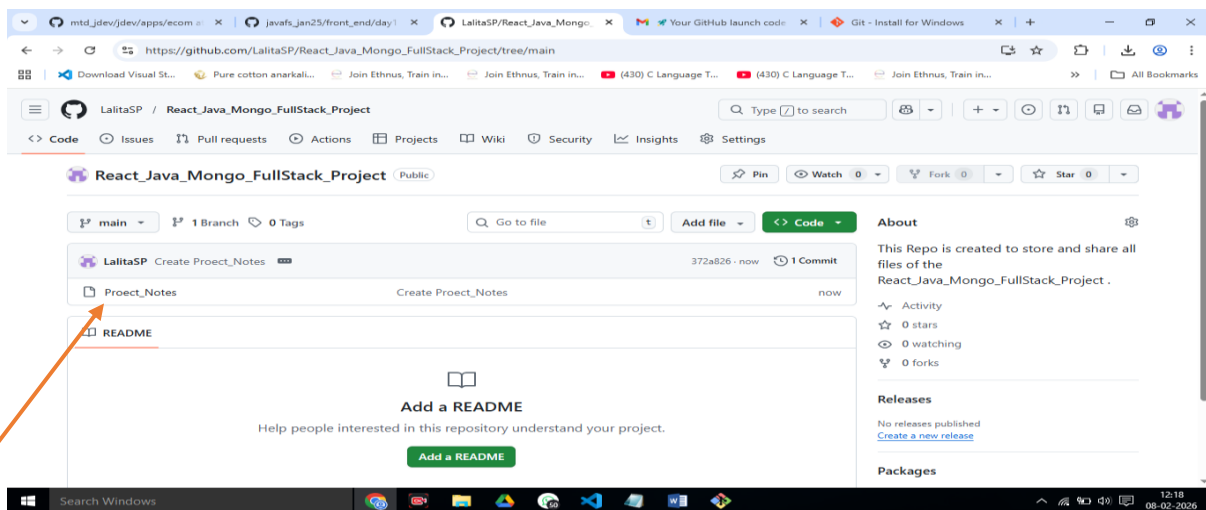
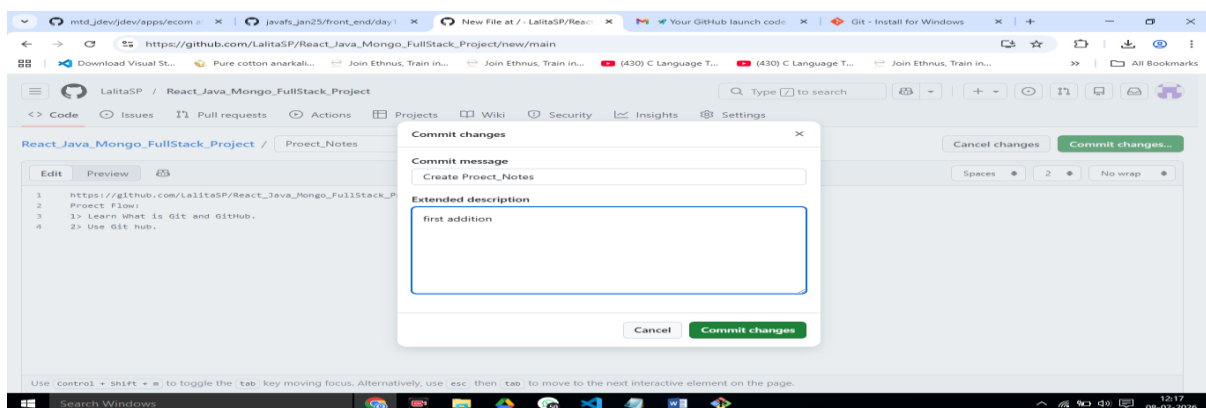
2> You can either initialize a new Git repository locally using `git init`

1> reate a new repository on GitHub.com.

- Click on create **new proect**
- Provide details like name of the proect like
"React_Java_Mongo_FullStack_Project"
- Then create new file. Type something about project in it and give name to your file
"**Proect_notes**".
- **Click on "commit changes" button.**



Next



=====

Clone this project repo to your computer folder:

Step 1: Copy Your GitHub Repository URL

1. Open your GitHub repository in browser
2. Click the green “Code” button
3. Copy the HTTPS link

Example:

=====

Step 2: Open Folder Where You Want the Project

1. Go to the folder where you want to keep your project
Example: D:\Projects
2. Right-click inside the folder
3. Click “Open Git Bash here”

Now we can use Git Commands to use our git project repo.

Step 4: Clone the Repository

Real-Life Analogy :

Think of GitHub repo like a **Google Drive folder**
Cloning = **Downloading the full folder to your computer**

In Git Bash, type:

```
git clone https://github.com/your-username/your-repo-name.git
```

```
cloning to....
```

Paste your actual link here

Press **Enter then** This will create a folder with your **repo name** inside that **directory**

Step 5: Check if It Worked

Type:

```
ls
```

You should see your project folder name
Now open it:

```
cd your-repo-name <React_Java_Mongo_FullStack_Project>
```

then type

```
>ls
```

It will show the file you ceated in git repo.

**Congratulations you have successfully
CLONED Your Git Repo Project !!!!!**

=====

How to Add a New File to Git Repo (from Git Bash) ?

Assume you already cloned your repo and are inside the project folder.

Step 1: Go to Your Project Folder

```
cd your-repo-name  this is done already..
```

Step 2: Create a New File

Example (some text file):

```
$ touch notes.txt
```

On Windows, you can also create file normally using Notepad.

Step 3: Add Content to File

- Open and edit:

\$notepad notes.txt

- notepad will open : Type in file some text.
 - Save and close.
-

Step 4: Check Status (Very Important Habit)

git status

You'll see:

```
Untracked files:
  Notes.txt
```

Step 5: Add File to Git Staging Area

- **git add Notes.txt**

- Or add all files:using below command.
 - **git add .**
-

Step 6: Commit the File (Save Snapshot)

git commit -m "Added Notes.txt file"

The file will be added and below lines will be displayed:

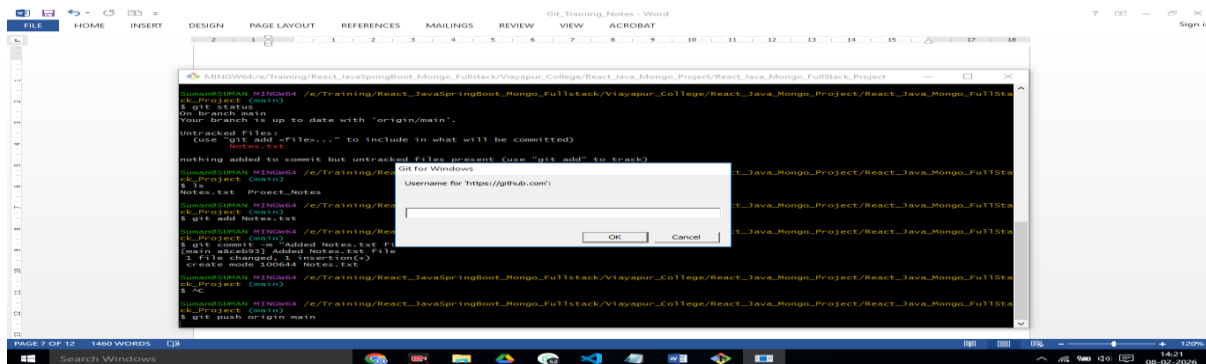
```
ck_Project (main)
$ git commit -m "Added Notes.txt file"
[main a8ceb93] Added Notes.txt file
1 file changed, 1 insertion(+)
create mode 100644 Notes.txt
```

Step 7: Push to GitHub

➤ `git push origin main`

Now refresh GitHub page — your file will be there !!!!!!!!!!!!!!!!

This command will authenticate for **pushes**: When you push your code to GitHub for the **first time**, a pop-up window will ask you to log in with your GitHub credentials. The Git Credential Manager will securely store your login token so you won't need to sign in again for future pushes.



GitHub Personal Access Token (PAT)

What is GitHub Personal Access Token (PAT)?

Think of PAT as a **special key** to your GitHub account — but safer than a password.

A **Personal Access Token (PAT)** is a **secure replacement for your GitHub password** when using:

- Git commands (push, pull, clone)
- VS Code Git sync
- APIs
- Automation (CI/CD)

It looks like this: It will look like: **ghp_XXXXXXXXXXXXXXXXXXXXXXXXXXXX**

Why Do We Use PAT Instead of Password?

GitHub **stopped allowing password login for Git operations.**

So this fails :

```
Username: yourname
Password: your GitHub password
```

You must use PAT instead :

```
Username: yourname
Password: ghp_xxxxxxxx (token)
```

Reasons GitHub uses PAT:

Reason	Explanation
➤ More secure	➤ Token can be limited to repo access only
➤ Can expire	➤ You can set expiry date
➤ Can be revoked	➤ If leaked, you can disable it
➤ Limited permissions	➤ Token can access only what you allow
➤ Safer for automation	➤ Used in scripts & CI tools

=====

When Do You Need to Use PAT?

You'll need PAT when:

Action	PAT needed?
<code>git push</code>	Yes
<code>git pull</code> (private repo)	Yes
Clone private repo	Yes
VS Code Git sync	Yes
Using GitHub API	Yes
Opening public repo	No
Cloning public repo	No (usually)

Use GitHub Token Instead of Password

Step 1: Create a GitHub Token (One-time)

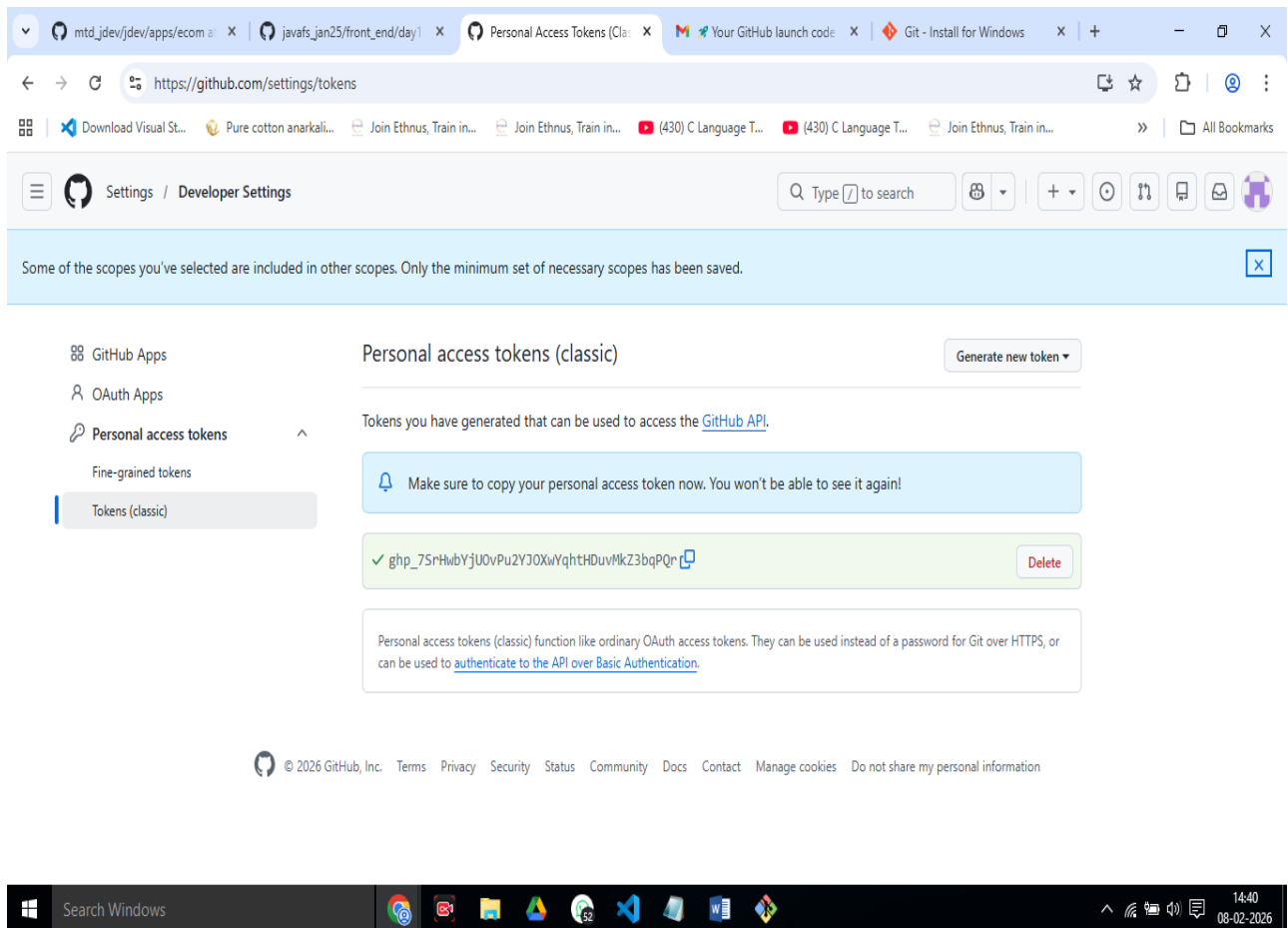
1. Open GitHub → Click your **profile pic (top-right)**
2. Go to **Settings**
3. Scroll down → **Developer settings**
4. Click **Personal access tokens** → **Tokens (classic)**
5. Click **Generate new token (classic)**
6. Give name:
7. **windows-git**
8. Expiry: choose **90 days** or **No expiration**
9. Tick this box only: **repo**
10. Click **Generate token**

It will look like: **ghp_XXXXXXXXXXXXXXXXXXXXXXX**

IMPORTANT:

Copy the token immediately (you won't see it again!)

Immediately send this **Token** to your **email** id instead of storing somewhere in file.



Step 2: Push Again (Use Token as Password)

Now in Git Bash:

```
$ git push origin main
```

It will ask:

Username: your-github-username

Password:

Enter:

- Username: <your name>
- Password: **PASTE THE TOKEN** (not your GitHub password)

It will push successfully

=====

Step 3: Save Login (So You Don't Repeat This)

Run:

```
git config --global credential.helper manager
```

Now Git will remember your token in Windows

=====

Why GitHub Did This?

Passwords are weak & unsafe for Git operations

Tokens are:

- ✓ more secure
- ✓ can be revoked
- ✓ limited permissions

=====

Most Important Git Commands (Cheat Sheet)

Command	Purpose
git --version	Check Git installed
git clone <url>	Download repo
git status	Check file status
git add <file>	Add file to staging
git add .	Add all files
git commit -m "msg"	Save changes
git push	Upload to GitHub
git pull	Download latest changes

Show commit history

Command	Purpose
<code>git log</code>	
<code>git diff</code>	See changes
<code>git branch</code>	Show branches
<code>git checkout <branch></code>	Switch branch
<code>git checkout -b newBranch</code>	Create new branch
<code>git rm <file></code>	Delete file
<code>git mv old new</code>	Rename file
<code>git remote -v</code>	Show repo URL
<code>git reset --hard HEAD</code>	Undo all changes (<u>A</u> dangerous)

Mini Example (Real Workflow)

```
touch main.c
notepad main.c
git status
git add main.c
git commit -m "Add main program"
git push origin main
```

How to Pull Files from GitHub to Your Local Folder?

(Pull = download latest changes from GitHub → into your PC)

Step 1: Go Inside Your Repo Folder

```
cd your-repo-name
```

(If you're already not there...)

Step 2: Check Current Status (Optional but smart)

```
git status
```

If you see:

```
working tree clean
$ git status
On branch main
Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean
```

You're good to pull.

Step 3: Pull Latest Code from GitHub

git pull origin main

This will download new files + updates made on GitHub or by teammates.

How to check this ?

Got to git repo folder in website. Type something in Notes.txt file and commit it..

=====

Example Output (Success)

```
Updating alb2c3..d4e5f6
Fast-forward
 new file:   api.js
```

Now refresh your folder — files will appear locally

When Do You Use `git pull`?

Use `git pull` when:

- ✓ You edited files directly on GitHub website
- ✓ Your teammate pushed new code
- ✓ You switched PC
- ✓ You want latest version before starting work

=====

Common Errors & Fixes

Error: "Your local changes would be overwritten by merge"

You changed files locally and didn't commit.

Fix (safe way):

```
git add .
git commit -m "Save my local work"
git pull origin main
```

OR discard local changes (A deletes your work):

```
git reset --hard
git pull origin main
```

Error: "Already up to date"

Super Easy Mental Model

```
git pull  = download updates
git push  = upload your changes
```

Like WhatsApp:

pull = receive messages

push = send messages

Mini Cheat Flow (Daily Routine)

```
git pull origin main    # before starting work
...code...
git add .
git commit -m "My changes"
git push origin main    # after finishing work
```

=====